

PRAMANA — Complete Work Breakdown

Smart India Hackathon 2026 · Problem Statement SIH26054 · DRDO Six people · Six days · One demo that has to work

How to read this document

Every person has a section. Your section tells you five things: **your mission in one line**, **what you own**, **exactly what you deliver and where it lives**, **what you do each day**, and **what you must not do**.

Read your own section closely. Skim everyone else's once, so you know who to ask when you get stuck.

The last section — *Contracts* — is the most important page in this document for the three developers. It is the set of promises we make to each other so that three people can build simultaneously without waiting on each other.

The one rule that beats every other rule

Build the twelve-hour version first. Finish it by end of Day 2. Tag it. Record a video of it.

Everything after that is *additive on a base that already works*.

Six days is a completely different game from a twelve-hour sprint, and the danger inverts. In a twelve-hour hackathon you lose by over-scoping. In six days you lose by **spreading** — ten things at 60% on the final day, and nothing that survives a full demo run end to end.

The Day-2 tag means we can never be caught with nothing. From Day 3 onward we are choosing to *improve a working system* rather than praying an unfinished one converges. The video recorded on Day 2 is our worst-case submission, and every later recording replaces it.

Day 5 evening is a hard feature freeze. No new code after that point, no exceptions, including the team lead's. Anything unfinished gets reverted to the last working commit — not "quickly fixed."

Day 6 is rehearsal only. Ten full runs, a different person driving each time.

Who owns which part of the problem statement

DRDO's expected solution is written as six lettered components, A through F. Evaluators score against a rubric derived from that list. Our folder names, slide headings and demo order all follow the same letters, deliberately — making the mapping obvious is close to free marks.

§	DRDO ASKS FOR	OWNER
A	Digital twin core, synchronised with live data, modular	BE-1
B	Health monitoring across the full channel set	BE-1
C	Fault detection incl. misfire, injector, lubrication, sensor drift	BE-1 + BE-2
D	AI/ML: anomaly detection, RUL, trend analysis, maintenance advice	BE-2
E	Simulation & replay, environment, endurance scenarios	Frontend
F	Visualisation dashboard	Frontend
—	Deployment roadmap <i>(a listed deliverable most teams ship zero slides on)</i>	R1

BE-1 — Physics & Simulation

Mission in one line

Build the engine. Everything the other five people do is downstream of a mean-value engine model that behaves like a real turbocharged aero-diesel, and of ten faults injected as physics rather than pasted onto a CSV.

Why your part is the moat

Almost every team that picks this problem will build a random-number generator feeding a CSV, an LSTM trained on it, and a dashboard with red and green lights. It demos fine. It loses, because a DRDO evaluator has seen forty of them by lunch and **none of them is a digital twin** — they are dashboards with a model bolted on.

The difference is entirely in your layer. When you inject a fault as a *parameter change* — injector discharge coefficient falling 0.4 %/min — the signature propagates through the physics on its own. Exhaust temperature on that cylinder rises, fuel flow drifts, the 0.5-order crank ripple appears, the energy balance shifts. **You never hand-author what a fault "looks like."** It comes out consistent for free, and every downstream model gets a physically coherent pattern instead of a spike someone drew.

You also get exact ground-truth RUL labels for free, which is the thing this entire field normally cannot get.

What you own

- **Layer 0** — the mean-value engine model (MVEM), the fault injector, the damage model
- **Layer 1** — CAN transport on a virtual bus, and the CAN→MQTT→WebSocket bridge
- **The parity residual generator** — ρ_1 through ρ_{11}
- **The UKF** joint state-parameter estimator
- **PS components A, B, C**

The five states you are modelling

You are building a *mean-value* engine model: cycle-averaged states, no resolution of individual crank angles. This is the correct fidelity, and you should be able to say why out loud. A crank-angle-resolved model would add in-cylinder pressure detail that **no flight sensor observes**, at a computational cost that would make both real-time operation and the Monte Carlo mission propagation impossible. Our MVEM runs several hundred times faster than real time on one core, and that speed is what makes the mission reliability layer affordable.

1. Intake manifold — filling and emptying

$$dp_{im}/dt = (R \cdot T_{im} / V_{im}) \cdot (\dot{m}_c - \dot{m}_a)$$

The compressor pushes air in, the cylinders pull it out. **Turbocharger degradation shows up here first**, as an inability to hold commanded boost.

2. Cylinder induction — speed-density, four-stroke

$$\dot{m}_a = \eta_v(p_{im}, N) \cdot p_{im} \cdot V_d \cdot N / (R \cdot T_{im} \cdot 120)$$

N in rpm, V_d swept volume. The 120 is two crank revolutions per cycle $\times 60$ s/min. **Volumetric efficiency η_v is the channel through which ring wear, valve-seat recession and induction restriction enter the model.**

3. Crankshaft dynamics

$$J \cdot d\omega/dt = T_{ind} - T_{fric} - T_{pump} - T_{load}$$
$$T_{ind} = \eta_i \cdot \dot{m}_f \cdot Q_{LHV} / \omega$$

Accumulate indicated torque **per cylinder**. This matters more than it looks: if you suppress one cylinder's contribution you get a *physically correct* misfire signature rather than an imposed one. In a four-stroke each cylinder fires once per 720° of crank rotation, so a single-cylinder defect appears at **0.5 engine order** in the angular-velocity spectrum — the classical misfire discriminant, recoverable from the crank position sensor that is already fitted.

4. Cylinder head thermal state — one per cylinder

$$m \cdot c_p \cdot dT_{cht,i}/dt = \dot{Q}_{gas,i} - h_{air} \cdot A_{fin} \cdot (T_{cht,i} - T_{cool})$$

Gas-side heat transfer follows a Woschni-type correlation, $h_c = 3.26 \cdot B^{-0.2} \cdot p^{0.8} \cdot T^{-0.55} \cdot w^{0.8}$, with B the bore and w a characteristic gas velocity.

Four independent thermal states are what make per-cylinder fault localisation physically meaningful instead of a colour someone assigned arbitrarily. Do not lump them.

5. Turbocharger shaft

$$J_{tc} \cdot \omega_{tc} \cdot d\omega_{tc}/dt = \eta_m \cdot P_{turb} - P_{comp}$$
$$P_{comp} = (\dot{m}_a \cdot c_p \cdot T_1 / \eta_c) \cdot [(p_2/p_1)^{(\gamma-1)/\gamma} - 1]$$

Scale η_c downward over time and you get compressor degradation that is invisible at sea level and becomes mission-limiting above critical altitude. That is a beautiful demo scenario and it costs you one multiplier.

Atmosphere — three lines that unlock the whole "environment simulation" ask

$$T_\infty = 288.15 - 0.0065 \cdot h \quad [K]$$
$$p_\infty = 101325 \cdot (1 - 2.25577e-5 \cdot h)^{5.2559} \quad [Pa]$$

Add a temperature offset for ISA+20 and you have the hot-and-high Leh case. Above the engine's critical altitude — **11 000 ft for the VRDE unit** — the turbocharger can no longer hold rated manifold pressure and power lapses. The twin reproduces this **without special-casing it**, which is the whole point of being physics-based.

Degradation — this is what turns a data generator into a labelled dataset

Wear accumulates through physically motivated laws, never injected noise. Carry an explicit scalar damage state $D \in [0,1]$ per component:

Adhesive/abrasive wear (Archard): $dD/dt \propto k \cdot F_n \cdot v / H$
Thermally activated ageing (Arrhenius): $dD/dt \propto \exp(-E_a / (R \cdot T_{cht}))$
Cumulative damage (Miner): $D = \sum n_i / N_i$, failure at $D = 1$

Ground-truth RUL = $(1 - D) / (dD/dt)$

Because you own `D`, every generated sample carries an exact RUL label with no censoring. This is the same construction NASA used to produce the C-MAPSS turbofan datasets, which have been this field's benchmark since 2008. Say that out loud when anyone asks about synthetic data — it is a methodological choice with eighteen years of precedent, not a limitation to apologise for.

The ten faults

Each is a parameter perturbation with its own time evolution. Because they act *through* the physics, the signatures come out consistent for free.

FAULT	INJECTED AS
Injector fouling, cyl <i>i</i>	<code>C_d ↓ 0.4 %/min</code>
Ignition misfire, cyl <i>i</i>	skip combustion event
Piston ring wear / blow-by	<code>η_v ↓</code> , blow-by ↑
Oil leak / pump wear	pump gain ↓
Turbo degradation	<code>η_c ↓ 8 %</code>
Cooling / intercooler fouling	<code>h·A ↓ 15 %</code>
Detonation / pre-ignition	knock model armed
Fuel filter clog / vapour lock	rail pressure ↓ with altitude
Bearing wear	friction torque ↑
Sensor drift, channel <i>x</i>	bias ramp on that channel only

That last row is the most valuable one in the table, and it is the cheapest to implement. Read it against the first row: an injector clog moves five residuals in a physically coupled pattern; a drifting EGT sensor moves **one**, with nothing corroborating it. That contrast is the demo beat at 3:00 that wins the room.

The residual generator

Produce `p1–p11` exactly as specified in `docs/spec/residual-spec.md`. Two things to be careful about, because both are places an expert evaluator will check:

- 1. **With *n* estimates of one quantity, only *n*–1 residuals are independent.** The third pairwise difference is the sum of the first two. A system claiming six residuals from four estimates is double-counting.
- 2. **Path 4 (intake restriction) does not exist on an unthrottled FADEC aero-diesel.** Return `null`, do not fabricate it. The engine YAML declares availability in `parity_paths` — read it, don't hardcode.

The CAN layer — 30 minutes, disproportionate payoff

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
pip install python-can paho-mqtt
```

Publish telemetry as **actual CAN frames** and decode them on the other side. The problem statement names CAN explicitly — not MQTT. Real UAV engine ECUs speak CAN, and having `candump vcan0` scrolling in a corner of the demo screen does more for credibility than any slide, because it says *this speaks the bus the ECU speaks*.

Name-drop the two real conventions in one sentence each: **CANaerospace** (Stock Flight Systems, 1998; adopted by NASA as the AGATE databus standard in 2001; identifiers 300–1799 reserved) and **DroneCAN/Cyphal** (what ArduPilot and PX4 actually run, 1 Mbit/s typical).

Your deliverables

FILE	BY
backend/twin/mvem.py — five states, integrator	Day 1
backend/twin/atmosphere.py — ISA + offset	Day 1
backend/twin/profiles.py — loads the engine YAML	Day 1
backend/twin/faults.py — all ten, as parameter perturbations	Day 2–3
backend/twin/damage.py — Archard / Arrhenius / Miner, D and RUL labels	Day 3
backend/parity/residuals.py — ρ_1 – ρ_{11}	Day 2
backend/estimation/ukf.py — joint state–parameter	Day 3–4
backend/transport/can_bridge.py — vcan0 → MQTT	Day 5
data/healthy_flights/*.parquet — the healthy training set	Day 2
data/fault_runs/*.parquet — labelled runs, all ten faults	Day 3

Definition of done, per day

- **Day 1** — the model runs, produces plausible numbers at a fixed operating point, and publishes over MQTT. Not accurate yet. *Running*.
- **Day 2** — thermal and turbo states in, six faults working, healthy dataset generated and handed to BE-2. **This is the Day-2 tag.**
- **Day 3** — all ten faults, damage integrator with ground-truth RUL labels.
- **Day 4** — mission forward-propagation, Monte Carlo over the degradation posterior.
- **Day 5** — CAN on vcan0, `candump` in the demo.

What you must not do

- Do **not** build a crank-angle-resolved model. Wrong fidelity, and it will not run fast enough for the mission layer.
 - Do **not** inject faults as signal spikes. The entire value is that they propagate through physics.
 - Do **not** change the telemetry schema without telling BE-2 and the frontend. A silent rename on Day 3 costs more than the feature was worth.
 - Do **not** tune the model to make a demo look good. If the physics says something, the demo shows it.
-

BE-2 — Machine Learning

Mission in one line

Three models, each with one job, each explainable in a sentence. Resist building five model families because the problem statement mentions several.

The single most important design decision in your layer

Everything you build eats the residual vector ρ , never raw sensor values.

This is not a stylistic preference. An autoencoder trained on raw signals learns the *flight profile* and screams every time the throttle moves — that is the single biggest reason naive builds false-alarm constantly, and it is the failure mode that will sink most of the other forty teams.

The residual vector has three properties raw telemetry does not: **its nominal value is zero, it is operating-point invariant by construction, and its noise distribution is characterised once on healthy data and reused everywhere.** So your autoencoder learns a genuinely simple target — *healthy means near-zero and white* — and does not need to relearn the flight envelope.

You should be able to say that paragraph from memory. It is the answer to at least two of the judges' questions.

M1 — Physics-guided residual estimator

Job: predict what every sensor *should* read at this operating point, plus the quantities we cannot measure at all.

```
 $\hat{y} = \underset{\text{MVEM}}{f_{\text{physics}}(u; \theta)} + \underset{\text{small correction}}{g_{\text{NN}}(u)}$ 
```

The MVEM does the heavy lifting; a small MLP learns only the **discrepancy**. Because the network learns a delta rather than the whole function, it trains on very little data and **cannot produce physically absurd predictions at operating points outside its training set** — which matters enormously, because your training data will be dominated by mid-altitude cruise and the aircraft flies to 28 000 ft.

Soft physical constraints in the loss:

```
 $L = L_{\text{data}} + \lambda_1 \cdot \underset{\text{energy}}{\|\rho_4\|^2} + \lambda_2 \cdot \underset{\text{mass continuity}}{\|\dot{m}_a + \dot{m}_f - \dot{m}_{\text{ex}}\|^2} + \lambda_3 \cdot \underset{\text{monotonicity}}{\|\min(0, \partial T_{\text{cht}} / \partial \dot{m}_f)\|}$ 
```

That third term costs one gradient penalty and buys sane behaviour outside the training envelope.

Virtual sensors this gives you — and DRDO asks for virtual sensing specifically, so these are the payoff: peak cylinder pressure, knock margin, per-cylinder combustion efficiency, turbocharger shaft speed, oil film thickness.

On PINNs, honestly. Do the *soft* version, not the hard one. A true PINN enforcing PDE residuals by collocation is a research project that may simply not converge inside our window, and if it doesn't we have nothing. What we claim — a physics backbone with a learned correction and physically motivated loss terms — is what we deliver, and we say so in the document. That distinction should be stated rather than blurred; the strict formulation offers little additional value for a lumped-parameter ODE system anyway.

M2 — LSTM autoencoder on residuals

Job: flag *this does not look like a healthy engine* without being told what the fault is.

Train on windows of ρ from healthy operation only. Threshold is the **99.5th percentile of reconstruction error on held-out healthy data** — never a hand-picked constant, and be ready to say that. Add an **N-of-M persistence rule** to suppress transients.

M3 — Isolation and remaining useful life

Classifier: 1D-CNN or GRU over the residual window $\rightarrow$ softmax across the ten fault classes. Labels are free because BE-1 generated the faults.

Run it **alongside** the incidence-matrix match from the residual spec, and show both agreeing. The network supplies accuracy; the matrix supplies the explanation. **Agreement between two independent mechanisms is itself evidence**, and judges remember the *why*.

RUL — two heads, always both:

- a quantile-regression network reporting **p10 / p50 / p90**
- a physics head that integrates the identified damage rate forward

Report both. "*Physics says 4.1 hours, the network says 3.7, we advise on 3.7.*" Disagreement between them exceeding the predictive interval is surfaced as a warning in its own right.

The UKF — with BE-1

Degradation parameters go **inside** the estimator's state vector as a slow random walk:

```
 $\theta = [ \eta_v\_scale, \eta_c\_scale, (hA)\_scale, C_{d,inj}^{(1..4)}, f\_fric ]$ 
```

Two consequences. The twin stays synchronised with a physically ageing engine without manual recalibration. And — this is the point — **the parameter estimates are themselves the health indicators, and they are physically interpretable.**

"Compressor efficiency scale 0.87, down from 0.94, standard deviation 0.02" is a statement a propulsion engineer can accept or dispute. "Health score 0.71" is not.

The filter also gives the rigorous form of sensor-fault discrimination. The innovation $v = y - h(\hat{x})$ measures surprise. **A component fault produces innovation that is absorbed by an adjustment of θ . A transducer fault produces innovation on one channel that no physically admissible parameter change can explain.**

The highest-value 90 minutes of your entire week

Run your residual → autoencoder → RUL stack, unmodified, on NASA's C-MAPSS or N-CMAPSS data, and put the score on a slide.

It is the only thing that proves our ML is not simply memorising our own simulator. Both datasets live in the NASA Prognostics Center of Excellence repository — "Turbofan Engine Degradation Simulation" is C-MAPSS, "...Simulation-2" is N-CMAPSS, which is the better one because it carries real flight conditions.

Do this on Day 1, before you touch our own data. Get a baseline number on somebody else's dataset first. It de-risks the entire week, and it means that if everything else goes wrong you still have the single most credible slide in the deck.

Metrics to put on the slide

RMSE alone looks like a course assignment. Add the field's own measures:

- **NASA asymmetric scoring function** — $\sum e^{(-d/13)} - 1$ if $d < 0$, else $\sum e^{(d/10)} - 1$. Late predictions punished harder than early ones. Field standard.
- **Prognostic horizon** and **α - λ accuracy** — standard PHM measures; using them signals familiarity with the discipline.
- **Detection lead time** — the operational value proposition.
- **False alarms per flight hour** — *the question a maintenance officer asks first*. Quoting it unprompted is a strong signal.

The ablation — one afternoon, proves our central claim

Same autoencoder, trained on **raw signals** vs trained on **residuals**. One bar chart. It converts our main design claim from an assertion into a measured result. Do it on Day 4.

Your deliverables

FILE	BY
ml/benchmarks/ncmapss_baseline.ipynb + a score	Day 1
ml/m1_residual_estimator/ — model + weights	Day 2
ml/m2_autoencoder/ — model, weights, calibrated threshold	Day 2
ml/m3_classifier/ — model + confusion matrix	Day 3
ml/m3_rul/ — both heads, p10/p50/p90	Day 3
ml/eval/metrics.md — lead time, FA/hr, ablation, N-CMAPSS score	Day 4
Pre-trained weights committed to the repo	Day 5

What you must not do

- Do **not** train on raw sensor values. Ever. Residuals only.
- Do **not** hand-pick an anomaly threshold. Calibrate it on healthy held-out data and be able to say how.
- Do **not** report a bare RUL number without its uncertainty band.

- Do **not** attempt a strict PDE-residual PINN. Scoped out deliberately, and we say why.
 - Do **not** train during the demo. Weights are committed.
 - **Do not build the GNN** unless Day 4 closed completely clean — and then only as a second isolation head *next to* the signature matrix, never as a replacement for it.
-

Frontend — Ground Control Station

Owned by the team lead. Written down here so it is not forgotten and so everyone knows what to expect on screen.

Mission in one line

Make it look like something in a ground control station, not like a SaaS analytics product — and make the 3D engine real, interactive, and bound to live health data.

What it owns

Layer 5, plus PS components E (simulation & replay) and F (visualisation dashboard).

Stack

Vite + React + TypeScript, Tailwind, `@react-three/fiber` and `drei` for the 3D, `uPlot` for strip charts, Zustand for state.

The interactive 3D engine — build from primitives, never a downloaded model

Every instinct says find a nice engine GLB on Sketchfab. **Don't.** A downloaded model arrives with a few hundred unnamed meshes, and hours disappear in the console working out which one is cylinder two. It is also ~40 MB and it drags the frame rate.

Compose it instead: four cylinders as `cylinderGeometry` with stacked torus fins, a crankcase as a rounded box, the turbo as a torus plus a cone, exhaust as tube geometry along a curve. About ninety minutes, **every mesh is a variable you named**, and binding health state to appearance becomes trivial:

```
<meshStandardMaterial
  color={thermalRamp(cht[i], 60, 135)}      // steel → amber → red
  emissive={faultColour(faultProb[i])}
  emissiveIntensity={0.15 + 0.85 * anomalyScore[i]} />
```

Animate the crank and pistons at scaled RPM. **That single touch is what makes people say "twin" instead of "diagram."** Orbit and zoom, and clicking a cylinder opens the explain drawer for it.

It will also look better, because it will look deliberate rather than like a stock asset dropped into a student project.

The rest of the screen

- **Strip charts: uPlot, not Recharts.** Recharts re-renders the React tree every frame and will visibly drop frames at 50 Hz with eight series. uPlot handles tens of thousands of points on canvas without complaint. This is the single most common performance mistake in hackathon dashboards.
- **Residual heatmap** — sensors down, time across, colour as deviation. One glance shows both *that* something is wrong and *which channels* are wrong. Most information-dense square on the screen, costs almost nothing to draw.

- **RUL with its confidence band always visible.** A bare number reads as overconfident to anyone who works with real hardware; a p10/p90 band reads as someone who has thought about it.
- **A timeline scrubber that drives everything** — charts, heatmap, and the 3D model. The PS asks for mission replay explicitly; making the scrubber *global* rather than a chart control is what turns a feature into a capability.
- **An explain drawer** — click any alert, see the residual contributions and the matched signature row. This is our interpretability story, and it is a UI panel, not a research problem.
- **Mission risk map** — route coloured by predicted risk, point-of-no-return as a moving marker, and the cruise altitude draggable so a judge can change the mission and watch the probability move.
- **Keep the limits panel visible the entire time** — what a threshold system would be showing right now. The contrast is the whole argument.

Aesthetics

Dark ground, tabular numerals everywhere, units labelled on every axis, no rounded pastel cards. Look at QGroundControl and Mission Planner for vocabulary, then make it about ten years newer.

Deliverables

ITEM	BY
Vite scaffold + one live chart off a mocked socket	Day 1
uPlot strips, 3D engine from primitives, health binding	Day 2
Explain drawer, residual heatmap, alert ribbon, replay scrubber	Day 3
Mission risk map, route colouring, point-of-no-return marker	Day 4
Visual polish, post-flight PDF, empty states	Day 5

Not blocked on anyone: mock the `pramana.health.v1` message from `docs/spec/telemetry-schema.md` and build the entire dashboard against it. When BE-1 and BE-2 are ready, swap the mock for the real socket and nothing else changes.

R1 — Research, Domain & Judge Readiness

Mission in one line

Become the person who can explain critical altitude and 0.5 engine order without notes, and make sure nobody on stage says anything factually shaky.

Over six days this is a real job, not filler. By Day 3 you should be able to hold a technical conversation with a DRDO evaluator unassisted.

What you own

- The problem-statement decode and the PS→repo mapping
- All domain research and source citations
- **The judge Q&A sheet** — the single highest-value document you produce
- **The deployment roadmap slide** — a listed DRDO deliverable most teams ship zero slides on

Why the deployment roadmap matters more than it sounds

A DRDO panel are **procurement people as much as engineers**. They are evaluating whether this could become a programme, not just whether it runs. A deployment roadmap is explicitly listed as a deliverable in the problem statement, and almost nobody will bring one.

PHASE	TRL	ACTIVITY	EVIDENCE PRODUCED
Current	3→4	Software twin, synthetic generation, internal consistency + external benchmark validation	Parity spread, N-CMAPSS score, isolation confusion matrix
Next	4→5	Recalibration against instrumented dynamometer data; seeded-fault testing	Detection lead time and false-alarm rate on real hardware
Then	5→6	Edge deployment on an airborne computing module; flight-test integration; recorded-data shadow mode	Latency, resource envelope, agreement with maintenance findings
Mature	6→7	Fleet analytics; maintenance-record feedback closing the diagnostic loop	Continuously improving priors from confirmed outcomes

That last row is worth a sentence of its own on the slide. It names the gap every deployed prognostic system in service today still suffers from: a diagnosis is issued, the aircraft lands, maintenance opens the component, and **the finding never returns to the algorithm**. Designing that feedback path in from the beginning is inexpensive now and effectively impossible to retrofit later. Saying that tells a procurement panel we have thought past the demo.

The alignment point — say this in the pitch

DRDO has issued a request for proposals under the **Technology Development Fund** for a digital twin framework for aero engine health monitoring, integrated with Health and Usage Monitoring Systems, structured in two phases. PRAMANA is scoped as a Phase 1 contribution, on the reciprocating rather than the gas-turbine side.

This is not a student exercise adjacent to an organisational interest. It is a prototype of a capability the organisation has formally solicited.

Facts you must have cold

- **VRDE 180 hp aero-diesel** — DRDO/VRDE, holds 180 hp to 11 000 ft, indigenously developed FADEC, 1100+ hours on test profiles, trialled at Leh and Changla to 17 664 ft.
- **TAPAS BH-201** — twin-engined, 220 hp each in production spec against 100 hp prototype units, demonstrated 28 000 ft ceiling and 18 h endurance, targeting 30 000 ft and 24+ hours.
- **Critical altitude** — the altitude above which the turbocharger can no longer hold rated manifold pressure and power begins to lapse. 11 000 ft for the VRDE unit. The aircraft operates to 28 000 ft, *far above it*, which is why compressor condition dominates available power on station.
- **0.5 engine order** — in a four-stroke, each cylinder fires once per 720° of crank rotation, so a single-cylinder defect shows at half engine order in the crank angular-velocity spectrum. Standard automotive misfire detection since the early nineties.
- **The Rotax 914 TCDS publishes no EGT limit.** EGT on these engines is a trend parameter, not a redline parameter. A threshold system has no way to use it. This is precisely the information a twin exists to exploit.
- **C-MAPSS** — NASA's synthetic turbofan degradation dataset, the field's benchmark since 2008. Synthetic. Our answer to "your data is fake."

The Q&A sheet — draft by Day 3

Rehearse these until they are reflexes. **Whoever on the team is weakest technically should be able to answer the first three.**

1. Your data is synthetic. Why should we believe any of it?
2. What does this give us that a threshold alarm doesn't?
3. How is this a digital twin and not a dashboard with a model behind it?
4. Which engine is this, and does it transfer to another?
5. How does this get onto an actual aircraft?
6. What's your false alarm rate?
7. Why should we trust an RUL number from a neural network?
8. What did you *not* build, and why?

Question 8 is a trap that we turn into an advantage — see *Honesty policy* below.

Deliverables

ITEM	BY
docs/research/research-notes.md — PS decode, org context, all citations	Day 1
docs/research/ps-mapping.md — A-F → deliverable → owner → file path	Day 1
docs/research/qa-sheet.md — all eight questions, answers written out	Day 3
Deployment roadmap slide content	Day 4
Fact-check pass over the entire deck and script	Day 5

R2 — Pitch, Deck & Narrative

Mission in one line

Own the story and the backup. The deck, the script, and — every single day, without exception — a fresh recorded video of whatever currently works.

The daily video is not optional

At the end of every day, record a full run of the current demo. It takes fifteen minutes. It means:

- We always have a submittable artefact, from Day 2 onward
- If the venue wifi dies, or a laptop dies, or a service won't start, we still present
- We can see our own progress, which matters more than people expect on Day 4

The Day-2 video is our worst-case submission. Every later recording replaces it. Verify each one plays back **offline** before you call it done.

How the pitch opens

Beat the threshold system on screen, not by describing our stack.

DRDO's own background paragraph is the most useful sentence they wrote, and most teams skip it: "*Conventional engine monitoring systems used in UAVs are primarily threshold-based and reactive in nature.*" That is DRDO telling us exactly what to beat. Open by doing it live.

Do not open with an architecture diagram. Do not open with the tech stack. Open with a fault developing while every light stays green.

The single chart that wins the room — show it before the architecture

A slowly clogging injector on cylinder 2. Exhaust gas temperature creeping up. Two systems watching it. **The redline system is silent for fifteen minutes. The twin is not**, because the twin knows EGT on cylinder 2 has no business being 12 °C above its three siblings at this fuel flow and this altitude.

- Twin alert at **t+3.7 min** — residual crosses 3σ and persists
- Threshold alert at **t+18.6 min** — EGT finally reaches 850 °C
- **14.9 minutes of warning** — enough to derate, divert, or land

Caption it honestly: "*Simulator output, not a claim about a physical engine.*" Baseline 720 °C, fouling ramp 7 °C/min, residual noise $\sigma = 4$ °C, alert requires 3σ sustained over a 2-minute persistence window.

That honesty note matters more than you would think. **Volunteering the limits of our own data is how we stop a judge from "catching" us with it**, and it costs nothing, because the mechanism is real even where the numbers are synthetic.

Deck structure

#	SLIDE	CONTENT
1	The problem, in DRDO's own words	Their background sentence, quoted
2	Why thresholds fail	Late by construction · blind to context · cannot doubt their own sensors
3	The chart	14.9 minutes of warning
4	The thesis	Over-determination — health is the <i>pattern of disagreement</i>
5	Architecture	Six layers, and where the simulation boundary sits
6	The physics	Five states, ten faults injected as parameters
7	Fault isolation	The incidence matrix — derived, not hand-authored
8	Sensor vs engine	The discriminator, and the demo beat
9	AI/ML layer	Three models, and why they eat residuals
10	Mission reliability	Continue / derate / RTB with probabilities
11	Validation	Three levels, including N-CMAPSS on someone else's data
12	Measured results	Lead time, false alarms/hr, ablation, confusion matrix
13	Deployment roadmap	TRL phases — R1 owns the content
14	What we deliberately did not claim	The honesty slide
15	Team & PS component mapping	A–F, literally

The honesty slide — slide 14, and it is not modesty

We state plainly: the techniques are established and we say so; the strict-PINN formulation is not used and we give the reason; three fault pairs are not structurally isolable from steady-state parity and we name them rather than hide them; cross-engine invariance is first-order and needs a small bias correction; parity residuals are still thresholded.

An evaluator who finds a limitation we concealed discounts everything else in the presentation. An evaluator who finds every limitation already identified and bounded reads the remaining claims as reliable.

In a technical evaluation that trade is strongly favourable. This slide makes us *more* credible, not less.

Deliverables

ITEM	BY
Deck skeleton, all 15 slide titles	Day 1
v1 backup video	Day 2
Slides 1–5 complete	Day 3
Full deck complete, narration written	Day 4
Deck done, rehearsal-ready	Day 5
Final video, verified playable offline	Day 6
<i>A fresh video, every single day</i>	<i>Daily</i>

R3 — QA, Repo & Scope Discipline

Mission in one line

You are the only person with authority to say "that's not in the demo script, don't build it." Over six days that role is worth more than another developer.

Use it. The team will thank you on Day 6 and resent you on Day 4, and that is the correct order.

What you own

- Repo hygiene, CI, and the `make` targets
- The status tracker — the single source of truth for *what actually works*
- QA passes and bug triage
- **Scope veto**
- Stopwatch on every rehearsal

The failure mode you exist to prevent

Around Day 4 somebody will want to add a transformer, or swap the autoencoder for something newer, or rebuild the frontend "properly." That urge is what six days does to people — it is the day the team most wants to start something new, precisely because the foundation finally works.

The answer is **no**. Day 4 is *measurement day* precisely for this reason: giving the team numbers to produce is the cheapest way to absorb that energy into something a judge will actually reward.

The `make` targets — build these on Day 1

Everything must be reproducible from one command. Nobody should be typing five commands under pressure on demo day.

```
make setup      # install everything
make sim        # run the simulator, publish to the bus
make edge       # residual generator + anomaly detector
make ui         # frontend dev server
make demo       # THE ONE THAT MATTERS – full scenario, end to end
make data       # regenerate all datasets
make test       # whatever tests exist
```

`make demo` is the most important line of code in the repository. It must work from a clean clone, offline, on someone else's laptop.

The status tracker

One row per feature, updated continuously. Status is one of: *not started* · *in progress* · *working* · *broken*.

The rule: **nothing is listed as "working" that you have not personally clicked through**. Not "the dev says it works." You clicked it.

This document is what R2 needs to know what the demo can honestly claim, and it is what stops us promising something on stage that quietly broke on Day 4.

Testing priority — in strict order

- 1. The exact demo path, in order. Nothing else matters as much.
- 2. The demo path with a judge doing something unexpected (clicking out of order, dragging the altitude slider to an extreme, refreshing mid-run)
- 3. A second browser and a second machine — judges will do this
- 4. Everything else

From Day 5 evening, **test only the demo path**. There is no time to fix anything else, so finding it only creates anxiety.

Deliverables

ITEM	BY
Repo structure, CI, all <code>make</code> targets	Day 1
<code>docs/qa/status-tracker.md</code> — live from Day 1, updated continuously	Daily
First end-to-end dry run	Day 2
<code>docs/qa/edge-cases.md</code>	Day 2
Second dry run + bug triage	Day 4
Third dry run, stopwatched	Day 5
<code>docs/qa/known-issues.md</code> — anything broken we will consciously avoid on stage	Day 5
Ten full rehearsal runs	Day 6

Bug triage rule

Every bug gets one of two labels: **breaks the demo path** or **doesn't**. Demo-path bugs get fixed first, always, regardless of how interesting the other one is.

Contracts — the promises that let us work in parallel

This is the most important page in this document for the three developers.

Contract 1 — BE-1 → everyone: the telemetry schema

`docs/spec/telemetry-schema.md` is **frozen**. Three lanes: fast (5–20 kHz, on-board only), slow (10–50 Hz, downlinked), health (1 Hz, downlinked).

Changing a field name or type mid-build requires telling both other developers *before* pushing. Adding a field is free. **A silent rename on Day 3 costs the team more than the feature was worth.**

Contract 2 — BE-1 → BE-2: the residual vector

BE-1 guarantees ρ_1 – ρ_{11} as specified in `docs/spec/residual-spec.md`. BE-2 builds every model against that vector and **never touches a raw sensor value**.

This contract is what lets BE-2 start on Day 1 with synthetic residuals while BE-1 is still writing the MVEM.

Contract 3 — everyone → frontend: `pramana.health.v1`

The 1 Hz health frame is the frontend's only input. The frontend mocks it from Day 1 and builds the entire dashboard against the mock. **The frontend is never blocked on the backend.** When the real socket is ready, the mock is swapped out and nothing else changes.

Contract 4 — the engine profile

A second engine is a **config file, not a rewrite**. Anything engine-specific — geometry, limits, maps, which parity paths exist — lives in `config/engine_*.yaml`. No engine constant is hardcoded anywhere, by anyone.

This is what makes the cross-engine transfer demonstration possible, and that demonstration is one of the five questions we can answer that other teams cannot.

Contract 5 — provenance

Every number that reaches a slide carries its origin: **published** (citable — type certificate, DRDO release, open source), **derived** (computed from published figures), or **assumed** (our engineering estimate).

Anything marked *assumed* that appears on a slide must be **labelled as such out loud**. This is not pedantry — it is the single cheapest way to keep an evaluator's trust for the claims that actually matter.

The six days

DAY	FRONTEND	BE-1 (PHYSICS)	BE-2 (ML)	R1 / R2 / R3
0	<i>All six, no code.</i> Freeze the telemetry schema, the engine profile, and the demo script. Agree who owns which PS letter. This half-day pays for itself three times over.			
1	Vite scaffold, one live chart off a mocked socket	MVEM states 1–3 + ISA, publishing over MQTT	Pull N-CMAPSS, get a baseline number before touching our own data	R1: PS decode + citations · R2: deck skeleton · R3: repo, CI, make targets
2	uPlot strips, 3D engine from primitives, health binding	Thermal + turbo states, six faults, healthy dataset generated	M1 + M2, thresholds calibrated	R3: first end-to-end dry run · R2: record v1 video
2 pm	TAG v1 AND RECORD IT. A working vertical slice: simulate → stream → residuals → anomaly → dashboard. Not impressive yet. Finished. From here we cannot lose everything.			
3	Explain drawer, residual heatmap, alert ribbon, replay scrubber	Remaining four faults, damage integrator with ground-truth RUL	M3 classifier + signature matrix + two-headed RUL	R1: Q&A sheet · R2: slides 1–5
4	Mission risk map, route colouring, point-of-no-return	Mission forward-propagation, Monte Carlo over degradation	Measurement day: false-alarm rate, lead times, ablation, N-CMAPSS score	R3: second dry run + triage · R1: deployment roadmap
5	Visual polish, post-flight PDF, empty states	CAN on vcan0, candump in the demo	Edge latency benchmark; GNN <i>only</i> if Day 4 closed clean	R2: deck done · R3: third dry run, stopwatched
5 pm	HARD FEATURE FREEZE. No new code, no exceptions, including the team lead's. Anything unfinished gets reverted to the last working commit — not "quickly fixed."			
6	Rehearsal only. Ten full runs, a different person driving each time. Q&A drill until answers are reflexive. Final video recorded and verified playable offline. Laptops charged, everything mirrored locally, one teammate carrying a duplicate.			

Cut list

Cut without hesitation

- The graph neural network
- Real PDE-residual PINN training
- Any database that isn't Parquet or SQLite
- Auth, user accounts, multi-tenancy
- Docker Compose with six services
- A trained model for anything that doesn't need one

Never cut

- **The residual concept** — it is the whole thesis
 - **The sensor-drift discrimination demo** — it is the line they repeat to the next team
 - **Per-cylinder resolution in the 3D view**
 - **The RUL uncertainty band**
 - **The mission decision panel**
 - **The replay scrubber**
-

Before the demo, every time

- ☐ Every dataset **pre-generated**. Never generate during a demo.
- ☐ Model weights **pre-trained and committed**. Never train during a demo.
- ☐ Whole scenario reproducible from a single `make demo`
- ☐ **Full video backup recorded the night before**, verified playable offline
- ☐ Laptops charged, everything mirrored locally, one teammate carrying a duplicate
- ☐ `candump vcan0` scrolling in a corner of the screen from second zero

Assume the venue wifi will fail. At some point, it will.